SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

Course Code:	CSA10 / CSA1023	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 2 – **Agile Sprint Planning & User Story Workshop**
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	U. Kavya	**Reg. No.:**	192472114
Date:	08-10-2026	**Duration:**	60 minutes

Code of Conduct

I _U.Kavya (192472114)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _U. Kavya_

Annexure A – Agile Sprint Planning & User Story Workshop

Instructions to Students:

Each student is assigned an individual software project problem statement. Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

1. **Identify the Project Vision and Stakeholders**
   - Define the project's vision, objectives, and identify all key stakeholders involved.
2. **Create Epics and User Stories**
   - Break down the requirements into Epics and formulate well-structured User Stories using the format: *As a <user>, I want <feature>, so that <benefit>.*
3. **Define Acceptance Criteria**
   - Write clear and measurable acceptance criteria for each selected user story using the Given–When–Then format or equivalent.
4. **Prioritize the Product Backlog**
   - Organize and prioritize the product backlog based on business value, customer needs, and project dependencies.
5. **Plan the Sprint Backlog**
   - Select high-priority user stories for the first sprint and divide them into actionable development tasks.
6. **Estimate Effort Using Story Points**
   - Assign story points to each selected user story using an appropriate estimation technique (e.g., Fibonacci sequence or Planning Poker) and justify your estimates.
7. **Present the Sprint Goal and Expected Deliverables**
   - Define a clear sprint goal and describe the expected sprint deliverables, including the features to be completed at the end of the sprint.

Project Title: Container Security and Runtime Behaviour Monitoring Platform

1. Project Vision and Stakeholders

Project Vision:

To develop a secure platform that scans container images, validates configurations, monitors runtime behaviour, detects suspicious activities, and integrates security checks with CI/CD pipelines to support secure and scalable container deployment.

The platform aims to:

- Scan container images for vulnerabilities.
- Validate container configurations.
- Monitor runtime behaviour.
- Detect suspicious or abnormal activities.
- Integrate security gates with CI/CD.
- Support container orchestration and centralized logging.
- Provide incident-response and audit mechanisms.

These objectives directly follow the assigned problem statement.

Key Stakeholders:

Stakeholder	**Role**
Security Administrator	Monitors vulnerabilities and security alerts
DevOps Engineer	Integrates security checks with CI/CD
Developer	Fixes vulnerabilities and configuration issues
System Administrator	Manages containers and infrastructure
Security Analyst	Investigates suspicious runtime behaviour
Project Manager	Plans and monitors project progress
End User/Organization	Uses the platform for secure deployment

2. Epics and User Stories

Epic 1: Container Image Security

US1 : As a **Security Administrator**, I want to **scan container images for vulnerabilities**, so that **vulnerable images can be identified before deployment**.

US2 : As a **Developer**, I want to **view vulnerability details**, so that **I can fix security issues in the container image**.

Epic 2: Configuration Security

US3 : As a **DevOps Engineer**, I want to **validate container configurations**, so that **misconfigured containers can be prevented from deployment**.

Epic 3: Runtime Behaviour Monitoring

US4 : As a **Security Analyst**, I want to **monitor container runtime activities**, so that **abnormal behaviour can be detected**.

US5 : As a **Security Administrator**, I want to **receive alerts for suspicious activities**, so that **security incidents can be investigated quickly**.

Epic 4: CI/CD Security

US6 : As a **DevOps Engineer**, I want to **integrate security checks into CI/CD pipelines**, so that **unsafe container images are blocked before production**.

Epic 5: Logging and Incident Response

US7 : As a **Security Analyst**, I want to **view centralized security logs**, so that **I can investigate container security events**.

US8 : As a **Security Administrator**, I want to **maintain audit records**, so that **security activities can be tracked and reviewed**.

3. Acceptance Criteria

ID	User Story	Acceptance Criteria
US1	Scan container images	**Given** a container image is uploaded, **When** the scan starts, **Then** the platform should identify and report vulnerabilities.
US2	View vulnerability details	**Given** vulnerabilities are detected, **When** the user opens the scan result, **Then** vulnerability details and severity should be displayed.
US3	Validate configurations	**Given** a container configuration is provided, **When** validation is performed, **Then** configuration errors or security issues should be reported.
US4	Monitor runtime activity	**Given** a container is running, **When** runtime activity occurs, **Then** the platform should collect and display relevant behaviour information.
US5	Suspicious activity alerts	**Given** abnormal behaviour is detected, **When** the monitoring system identifies it, **Then** a security alert should be generated.
US6	CI/CD security gate	**Given** a CI/CD pipeline runs, **When** a security check fails, **Then** the deployment should be prevented or flagged.
US7	Centralized logging	**Given** security events are generated, **When** logs are collected, **Then** they should be available in a centralized view.
US8	Audit records	**Given** security activities occur, **When** an administrator checks the audit system, **Then** the relevant activities should be recorded.

4. Product Backlog Prioritization

Priority is based on **business value, security importance and dependencies**.

Priority	ID	User Story	Story Points
1	US1	Container image vulnerability scanning	5
2	US3	Container configuration validation	3
3	US4	Runtime behaviour monitoring	5
4	US5	Suspicious activity detection and alerts	5
5	US6	CI/CD security integration	8
6	US7	Centralized security logging	3
7	US2	Vulnerability details	3
8	US8	Audit records	2

Priority Justification

- **US1** is highest because vulnerability scanning is a core security function.
- **US3** is important because insecure configurations should be detected early.
- **US4 and US5** provide runtime protection.
- **US6** connects security with the development and deployment process.
- **US7 and US8** support monitoring, investigation and auditing.

5. Sprint Backlog – Sprint 1

Sprint Duration: 2 weeks

Selected User Stories:

ID	User Story	Story Points
US1	Container image vulnerability scanning	5
US3	Configuration validation	3
US4	Runtime behaviour monitoring	5
US5	Suspicious activity alerts	5
US7	Centralized security logging	3
Total		**21 points**

Calculation: Total Sprint Story Points

= US1 + US3 + US4 + US5 + US7

= 5 + 3 + 5 + 5 + 3

= **21 Story Points**

So, the Sprint 1 workload is **21 story points**.

6. Sprint Tasks

US1 – Vulnerability Scanning – 5 Points

1. Create image upload/input module.
2. Connect vulnerability scanning component.
3. Analyse container image.
4. Identify vulnerability severity.
5. Display scan results.
6. Test scanning functionality.

US3 – Configuration Validation – 3 Points

1. Create configuration input.
2. Define security validation rules.
3. Validate configurations.
4. Display configuration errors.
5. Test valid and invalid configurations.

US4 – Runtime Monitoring – 5 Points

1. Connect to running containers.
2. Collect runtime activity.
3. Monitor container behaviour.
4. Store monitoring data.
5. Display runtime information.
6. Test monitoring.

US5 – Suspicious Activity Alerts – 5 Points

1. Define abnormal behaviour rules.
2. Analyse runtime events.
3. Detect suspicious activity.
4. Generate alerts.
5. Display security alerts.
6. Test alert mechanism.

US7 – Centralized Logging – 3 Points

1. Collect security logs.
2. Store logs centrally.
3. Create log viewing interface.
4. Filter/search logs.
5. Test logging functionality.

7. Story Point Estimation:

We use the **Fibonacci sequence**: **1, 2, 3, 5, 8, 13**

Story points represent **relative effort, complexity and uncertainty**, not exact hours.

Story	**Points**	**Justification**
US1 – Vulnerability Scanning	**5**	Requires image processing, vulnerability detection and result generation
US3 – Configuration Validation	**3**	Moderate number of validation rules
US4 – Runtime Monitoring	**5**	Requires continuous monitoring and data collection
US5 – Suspicious Activity Alerts	**5**	Requires behaviour analysis and alert generation
US6 – CI/CD Integration	**8**	Complex integration with external CI/CD pipeline
US7 – Centralized Logging	**3**	Requires log collection, storage and display
US2 – Vulnerability Details	**3**	Requires displaying and organizing scan information
US8 – Audit Records	**2**	Relatively simple recording and retrieval functionality

Sprint Calculation

Sprint 1 = 5 + 3 + 5 + 5 + 3

Sprint 1 = 21 Story Points

The **8-point CI/CD integration** is kept for a later sprint because it is comparatively complex and depends on the basic security functions being available first.

8. Sprint Goal

Sprint Goal: "Develop the core container security monitoring functionality by implementing vulnerability scanning, configuration validation, runtime behaviour monitoring, suspicious activity alerts, and centralized security logging**."**

9. Expected Sprint Deliverables

At the end of Sprint 1, the team should deliver:

1. Container image vulnerability scanner.
2. Configuration validation module.
3. Runtime behaviour monitoring module.
4. Suspicious activity detection and alert system.
5. Centralized security logging.
6. Basic security dashboard/results view.
7. Tested and working core security features.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder	Project vision is clear, comprehensive, and aligned with the problem	Vision is clear with minor omissions. Most	Vision is partially defined. Some stakeholders are	Vision is unclear or incomplete. Stakeholders are

Identification (30%)	statement. All relevant stakeholders and their roles are correctly identified.	stakeholders are correctly identified.	missing or incorrectly identified.	poorly identified or missing.
Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	Mark
Project Vision & Stakeholder Identification **(30%)**	/5
Epics & User Stories **(25%)**	/5
Acceptance Criteria **(20%)**	/5
Product Backlog Prioritization & Sprint Planning **(15%)**	/5
Story Point Estimation, Sprint Goal & Presentation **(10%)**	/5
TOTAL	/25